

AI Usage Log: Sprint 1

SOEN 341: CareerConnect
Lamees Ibeid, Backend Team

AI Tool Used: Claude (Anthropic). All of my Sprint 1 AI use took place in a single conversation.

Chat Link (applies to all entries):

https://claude.ai/code/session_019DWxSTMhWGMoo7GaWLc4Xt

My Sprint 1 Work: Backend project setup, password hashing, the user registration endpoint and its tests, and connecting registration with the frontend.

Task 1: Backend Task Breakdown and Sprint 1 Planning

Purpose of AI Use: Planning, task allocation and requirements analysis

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Generated a breakdown of the Sprint 1 backend work split between Youssef and me (project setup, password hashing, the registration endpoint and the login endpoint), a Git branching workflow, and a step-by-step guide for each of us. The first version also included resume upload.

Validation: Compared the plan with the Sprint 1 requirements (two basic features) and with the scope approved by our TA. Discussed the split with Youssef and the team before starting.

Decision: Modified before use. Resume upload was removed because the TA approved registration and login as our two Sprint 1 features.

Reflection: At the start I did not know what Python, FastAPI or a backend endpoint really were, so having the work broken into small ordered steps made it possible for me to start at all. I learned that a plan from AI still has to be checked against what the TA and the team agreed on, since the first version included a feature we were not doing this sprint. The breakdown was very useful for dividing the work clearly with Youssef.

Responsible Person: Lamees Ibeid

Task 2: Backend Project Setup (Dependencies, Environment Files and Configuration)

Purpose of AI Use: Code generation, debugging and learning (virtual environments, pip, environment variables)

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Explained how to create a virtual environment and install the backend packages, added the packages needed for authentication and testing to requirements.txt, and diagnosed why the installation failed on my Windows laptop: the uvloop package does not support Windows, so it suggested a platform condition that skips it on Windows. It also suggested the .gitignore entries, a .env.example template and a configuration file that reads settings from .env.

Validation: Ran the installation on Windows successfully after the fix and checked my changes with git diff before committing. When my branch conflicted with the frontend branch on .gitignore and .env.example, I combined both teams' entries so nothing was lost. The pull request was reviewed and approved by a teammate before merging.

Decision: Modified before use.

Reflection: I learned what a virtual environment is, why .env files hold settings that should never be pushed to GitHub, and that a package can work on one operating system and fail on another. I made several beginner mistakes here, such as running the activate command multiple times, saving files with leading spaces and creating a file with the wrong name, and the AI helped me understand each error message instead of just giving me a fix. Resolving my first merge conflict was confusing at first, but I now understand why it happens when two branches edit the same file.

Responsible Person: Lamees Ibeid

Task 3: Password Hashing

Purpose of AI Use: Code generation and learning (why passwords are hashed)

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Suggested two functions: one that turns a password into a secure hash during registration, and one that checks a typed password against the stored hash during login, using the Argon2 algorithm. It explained why plain passwords must never be stored in the database.

Validation: Wrote and ran a test that checks that the hash is different from the password, that the correct password is accepted and that a wrong password is rejected. The test passed.

Decision: Modified before use. I rewrote the function descriptions so they state clearly which one is used for registration and which one is used for login.

Reflection: I learned that hashing is one-way, so even someone with access to the database cannot read the users' passwords. Rewriting the descriptions myself helped me understand where each function fits in the registration and login flow. The explanation was useful and the code was short enough for me to understand fully.

Responsible Person: Lamees Ibeid

Task 4: Backend Application Setup (CORS, Error Format and Health Check)

Purpose of AI Use: Code generation and learning (endpoints, CORS and status codes)

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Suggested the main application file with CORS enabled for our React frontend, a rule that converts input errors into a single readable message with status 400 (the format the frontend expects), a general error handler, and a simple health check endpoint to confirm the server is running.

Validation: Started the server and opened the health check and the automatic documentation page in the browser. The health test passed. I also checked that the error format matched what Krish's frontend code reads.

Decision: Accepted.

Reflection: I did not know what CORS was before this. I learned that the browser blocks the frontend from talking to a backend on a different port unless the backend allows it, which later turned out to matter when we connected the two. I also learned what the common status codes mean. The AI's explanations of basic terms like endpoint and schema were the most useful part for me.

Responsible Person: Lamees Ibeid

Task 5: Test Setup and Debugging

Purpose of AI Use: Test setup and debugging

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Suggested a pytest configuration and a shared test file that uses a temporary database, so tests never touch real data. It then helped me fix three errors: the tests could not find the App folder, the test client could not be found, and the database address was missing. The causes were that I had pasted file content into the terminal instead of a file, two files had typos in their names, and I had no local .env file.

Validation: Re-ran the tests after each fix until they all passed, and removed the incorrectly named files from the repository.

Decision: Accepted.

Reflection: I learned the difference between typing a command in the terminal and writing code in a file, which I had mixed up. I also learned that one small typo in a file name can stop the whole test setup from working. Reading the error message carefully usually pointed to the

problem, and the AI helped me learn how to read them.

Responsible Person: Lamees Ibeid

Task 6: Registration Endpoint

Purpose of AI Use: Code generation and requirements analysis

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Suggested the registration request format and the registration endpoint. It converts the email to lowercase, rejects an email that is already registered, hashes the password, saves the user, and replies with the new user without the password. The field names and the two roles (job seeker and recruiter) matched the registration format that Krish shared for the frontend.

Validation: Checked the fields against Krish's frontend code and the database team's user model, tested the endpoint on the documentation page, and ran all the tests. While comparing with the database model I noticed that the database stores names up to 50 characters while my code allowed 100. The pull request was reviewed and approved before merging.

Decision: Modified before use. The maximum name length was changed from 100 to 50 characters to match the database.

Reflection: I learned that the backend must check the data even though the frontend already checks it, because anyone can send a request directly to the backend. Catching the 100 versus 50 mismatch myself showed me that AI output has to be checked against the rest of the team's work, since the AI did not know the database limit. This was the most important part of my sprint and the AI made it manageable.

Responsible Person: Lamees Ibeid

Task 7: Registration Tests

Purpose of AI Use: Test case generation

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Suggested tests for a job seeker and a recruiter registering, the password never appearing in the reply, a duplicate email in any capitalization, an invalid email, a short password, a missing role, a blank name, missing fields, and the CORS rule. It also suggested a test for an invalid role such as ADMIN. When I asked to use my real name and personal email as the test data, it advised using a fake example email instead.

Validation: Read every test and added a comment explaining what it checks. All 12 tests passed on the main branch after my work was merged.

Decision: Modified before use. I rejected the invalid role test, and I used lamees@example.com instead of my real email.

Reflection: I decided the invalid role test was not needed, which showed me I can disagree with the AI's suggestions. I agreed with its advice not to put my personal email in a public repository. Adding a comment to each test forced me to understand what each one does and what assert means, which will help me explain the tests during the lab.

Responsible Person: Lamees Ibeid

Task 8: Reviewing the Database User Model

Purpose of AI Use: Code review

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Compared the database team's user model with what registration needed, pointed out that the first merged version did not yet match the fields and roles registration needed, and helped me write a message to the database team.

Validation: I checked the model on GitHub myself before contacting the team. After Sayf fixed it, I confirmed that the registration tests passed with the updated model.

Decision: Modified before use. I wrote the final message to the team myself.

Reflection: I learned that the database model and the backend code have to agree exactly or registration fails. Checking the code myself before messaging the team was important, because I did not want to report a problem I had not confirmed. This interaction also showed me how much coordination between sub-teams a feature needs.

Responsible Person: Lamees Ibeid

Task 9: Git Workflow, Commit Messages and Pull Requests

Purpose of AI Use: Learning and documentation writing

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Explained branches, commits, pushing, pull requests and the direction of a merge, how to resolve merge conflicts, and how to exit the Vim editor during a merge. It also drafted commit messages and pull request descriptions in the same style as Krish's pull requests.

Validation: Followed the steps on my own laptop. Both of my pull requests were reviewed and approved by teammates before being merged into main.

Decision: Modified before use. I asked for simpler, clearer wording for my commit messages and pull request descriptions and adjusted them.

Reflection: Before this sprint I had never used Git in a team. I now understand why each feature gets its own branch, why changes go through a pull request instead of straight into main, and how to update my branch when main changes. Writing my own commit messages in plain language made my history easier for the team to follow.

Responsible Person: Lamees Ibeid

Task 10: Connecting the Frontend and Backend

Purpose of AI Use: Debugging and research

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Found why registration showed "Not Found" and "Something went wrong" for my teammates. The frontend was calling `/api/auth/register` while the backend is at `/api/v1/auth/register`, as planned in the project documents, and Krish's fix was only on his branch and not merged into main yet. It also found that teammates were missing the local `.env` file with the database address, because the README had no backend setup steps.

Validation: Tested both addresses against the backend: the old address returned 404 (not found) and the new one returned 201 (created). I then registered a user from the frontend on Krish's fix branch successfully.

Decision: Accepted.

Reflection: I learned that "v1" means version 1 of the API, and that a single missing part of an address is enough to break the connection between frontend and backend. I also learned why it worked on my laptop but not on my teammates': I had a local `.env` file that is not shared on GitHub. This showed me that setup instructions in the README matter as much as the code.

Responsible Person: Lamees Ibeid

Task 11: Understanding the Sprint 1 Requirements and AI Log

Purpose of AI Use: Requirements analysis and documentation

Chat Link or Prompt/Response: Chat link provided above.

AI-Suggested Content: Summarized the Sprint 1 requirements and the AI log criteria from the project description, and helped format and write this log based on our conversation.

Validation: Compared the log with the AI Usage Log Requirements section and the example entry in the project description, and reviewed each entry against what I actually did during the sprint.

Decision: Modified before use. I asked for a simpler format that follows the example in the project description.

Reflection: Documenting each interaction made me realize how much of my learning this sprint happened through asking questions. Going back over the decisions reminded me where I accepted, changed or rejected the AI's suggestions, which will help me explain my AI use during the lab review.

Responsible Person: Lamees Ibeid